Section IV: Technical Approach

IV: Problem Formulation

As stated, the purpose is to essentially discover an approach to train a model to identify edges within an image to the extent that most humans would agree with the model on most occasions, without the use of expensive human-annotated supervised labels. In the abstract, we detail 2 separate experiments that use a slightly different pipeline and involve a model trained via weakly supervised approaches to see if they can outperform the lower bound of a Canny algorithm. Finally, we will compare the weakly supervised models to both raw Canny and the supervised SOTA model to fully comprehend whether a neural network can smooth noise and generalize further from strict algorithm-generated supervision to outperform the raw algorithm-generated results themselves and compete with detailed human-annotated supervision.

First, we provide a rigorous mathematical definition of the input, output, and how the models manipulates both of these. We define the input image space to be $X \in [0, 1]^{HW}$ where H is the height of the image (256 pixels) and W is the width of the image (256 pixels). This basically means an ordered list of real numbers between 0 and 1 with HW entries as there are HW total pixels and an entry x_i represents the pixel intensity or brightness at the i-th index. We define it as such since our PyTorch code eventually normalizes the brightness values into real numbers between 0 and 1 inclusive and we can imagine that the pixel at (0, 0) corresponds to the 0th index, (1, 0), corresponds to the 2nd index and the leap from (255, 0) to (0, 1) just corresponds to an increment (+1) with respect to the i-th index. The rest of the index correspondence can be understood analogously. In that case, the binary edge target can be defined as $Y \in \{0, 1\}^{HW}$ where a 0 at the i-th index indicates that there is no edge at the corresponding pixel and a 1 indicates that there is an edge at the corresponding pixel. Our intention is for our models to essentially to learn a non-linear mapping $f_\theta: X \rightarrow [0, 1]^{HW}$ parameterized by neural network weights θ where supervision is derived deterministically and algorithmically as opposed to human labelling. In both experiments, the supervised label is algorithmically derived from a simple Canny edge detection algorithm, and the parameters represented by θ will converge via backpropagation from the loss based on the Canny label.

Experiment 1:

In experiment 1, we use X_{raw} meaning that the input received by the model is just the raw grayscale image. Our learned model f processes it and returns for us ordered probabilities in which each probability at a specific index tells us the model's confidence that the pixel corresponding to that index is an edge pixel. This can be represented as $X_{raw} \rightarrow f_\theta \rightarrow \hat{Y}$.

Experiment 2:

In experiment 2, the direct input to the model is defined as $X_{lap} = \tau_{lap}(X_{raw})$. This means we are applying a laplacian transform on the raw grayscale image to further distinguish contrasting

regions before sending it to the model. The idea is to test if this will bolster the model's learning and help it converge faster and more optimally. The full pipeline can be represented as $X_{raw} \rightarrow \tau_{lap} \rightarrow X_{lap} \rightarrow f_\theta \rightarrow \hat{Y}$.

IV: Classical Priors & Pseudo-label curation

Canny Modifications:

Natural images contain many high-frequency micro-textures that trigger classical false positives. This must be addressed since, although we are purposefully using a weaker supervised label relative to human annotation, if the label uniquely captures an enormous amount of noise relative to the ground-truth label, then no amount of model generalization and smoothing will be able to salvage that. Invoking a Canny algorithm directly from the OpenCV function showcases to us the extent of the issue.

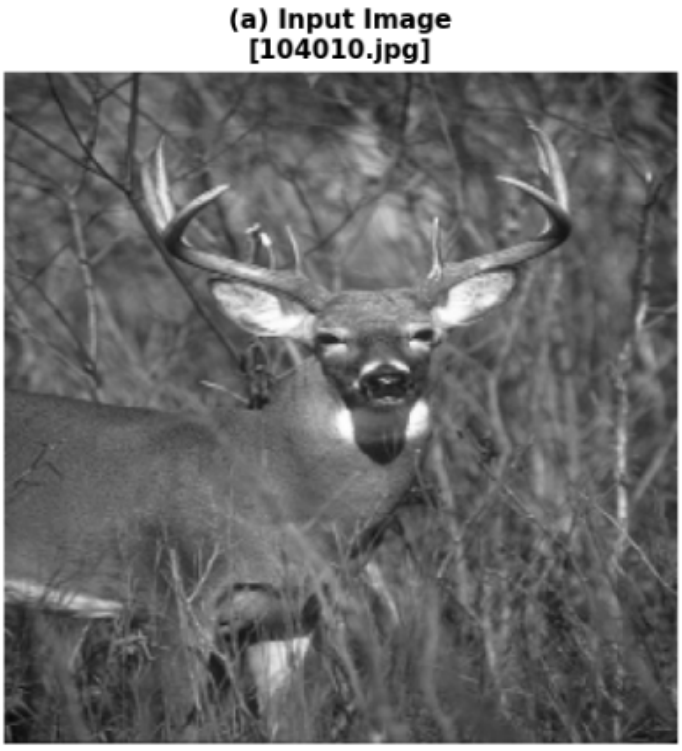


Fig. 1. An input image to model f_θ

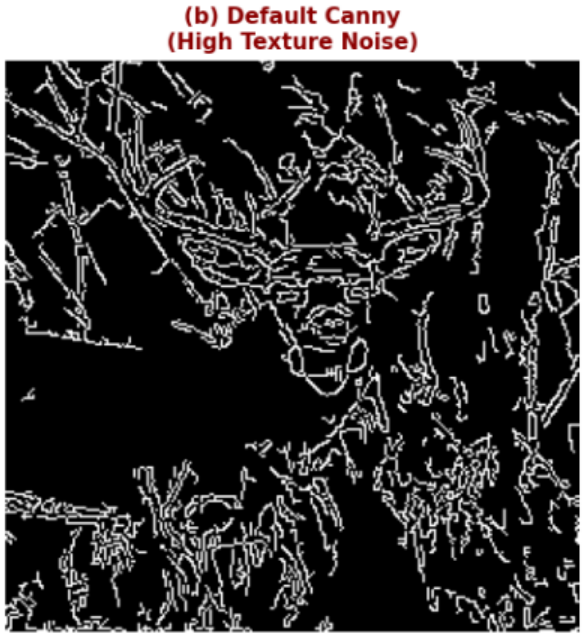


Fig. 2. Raw Canny edge detection (from OpenCV)

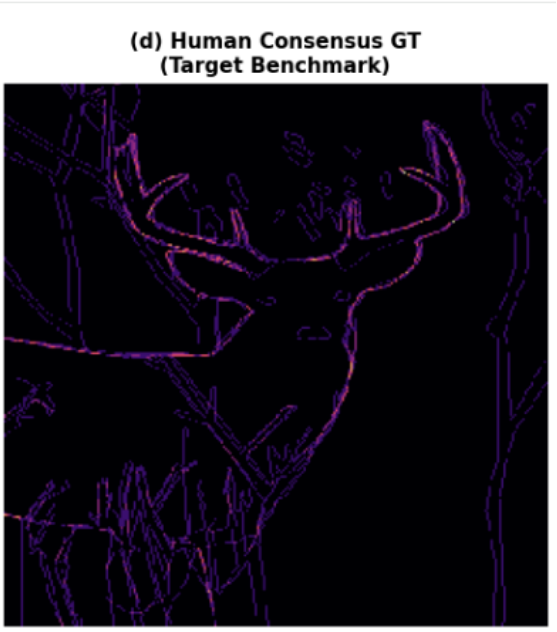


Fig. 3. Ground truth human consensus

From this, it is clear that the Gaussian smoothing applied by the default Canny algorithm is subpar. By increasing the variance and the window in which the averaging takes place, we can increase the influence of further away pixels in the averaging to increase smoothing. So, from the default of roughly $\sigma \approx 1.1$, we increase this to $\sigma = 1.5$ and instead of a 5 x 5 window for averaging, we increase this to a 7 x 7 window.

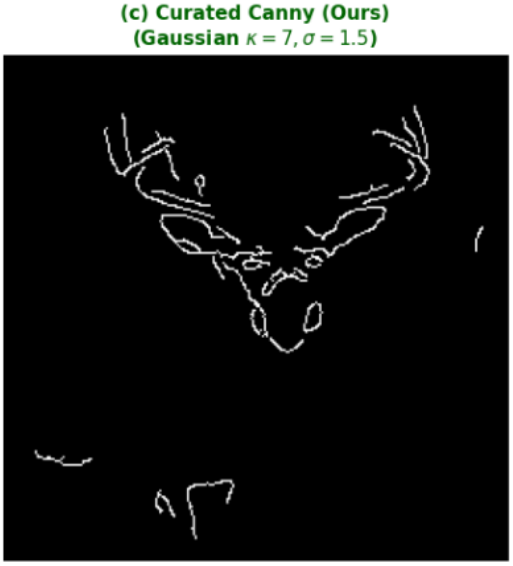


Fig. 3. Canny result after our custom Gaussian filtering. The curated Canny identifies edges that are much more aligned with the ground truth than the default Canny.

In the case of Laplacian, we mathematically demonstrate how the method works to sharpen images and bolster contrast, providing an extra nudge in the right direction to the model of our 2nd experiment. In a discrete image, Δx = Δy = 1 and thus the second partial derivatives along the horizontal and vertical axes are approximated using central differences:

$$\frac{\partial^2 f}{\partial x^2} \approx f(x+1,y) - 2f(x,y) + f(x-1,y)$$

$$\frac{\partial^2 f}{\partial y^2} \approx f(x,y+1) - 2f(x,y) + f(x,y-1)$$

This further gives us:

$$\nabla^2 f(x,y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\nabla^2 f(x,y) \approx \Big[f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1)\Big] - 4f(x,y)$$

Dividing both sides by 4 reaches:

$$\frac{1}{4}\nabla^2 f(x,y) \approx \underbrace{\frac{f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1)}{4}}_{\bar{f}_{\text{local}}(x,y)} - f(x,y)$$

$$\nabla^2 f(x,y)$$

Mathematically, this indicates that is directly proportional to the difference between the average of the top, bottom, left and right pixels and the central pixel. So the magnitude of the Laplacian gives us a direct indication of the level of difference between a central pixel and its surrounding pixels, excluding the diagonal ones relative to the center.

$$\mathbf{X}_{\text{lap}}(x,y) = \mathbf{X}(x,y) - \alpha \cdot \nabla^2 \mathbf{X}(x,y)$$

This leads to where α is a coefficient that controls the level of change to particular pixel that we want to have. The idea is that when a central pixel is very different from its neighbouring pixels, the magnitude of the Laplacian would be larger. When this difference is notable, the more we want to alter this central pixel to highlight this difference further, and thus by subtracting the central pixel from its corresponding Laplacian, we are able to boost the central pixel in a direction that emphasizes the difference when the difference was already notable. When the difference between the central pixel and the neighbouring pixels are minimal, the magnitude of the Laplacian would be close to 0, and thus

the central pixel would be barely altered, reflecting our desires of maintaining pixel values when the neighbouring pixels are similar. This leads to a situation where the contrast between edges is further amplified while pixels far from an edge are maintained. The mask below gives us a mechanism for computing the Laplacian of a central pixel. In our code, for α, we chose a value of α = 0.5 to ensure a moderate high-boost sharpening effect while mitigating the amplification of camera noise and pixel saturation.

$$K_{\nabla^2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

By applying Laplacian to entire grayscale image before training, hopefully, we will be able to ease the model's burden with respect to learning, as the edges of the input are amplified.

IV: Lightweight U-Net Architecture

Moving on to f_θ, our choice of model that learns the ideal f_θ for this role was a U-Net. Instead of a standard U-Net that has roughly 31M parameters, we design a custom lightweight variant with only 480K trainable parameters. The reason for this is that a model that is standard or too large relative to the size of the training sample will be very good at learning the intricate details of the supervised labels and will match it more precisely. Normally, this would be recognized as an ideal in machine learning. However, for our purposes, we are using Canny labels, which we already know to be flawed and noise-ridden, and our goal is to see if a model can generalize the underlying structure while neglecting the noise, so a 31M parameter model subject to only 200 training images will be more likely to overfit to intricate details corresponding to noise, leading to outputs that excessively resemble Canny, counteracting our goal. The network consists of four primary structural stages: the core convolutional building block, the contracting encoder path, the expansive decoder path with long-range skip connections, and a scalar projection head. As the fundamental feature-extraction unit, across both encoding and decoding stages, each module of the dual-convolutional model is structured as two consecutive sequences of a two-dimensional convolution, batch normalization, and a rectified linear unit (ReLU) activation.

$$\mathbf{F}_{\text{out}} = \text{ReLU}\left(\text{BN}\left(\text{Conv}_{3\times3}\left(\text{ReLU}\left(\text{BN}\left(\text{Conv}_{3\times3}(\mathbf{F}_{\text{in}})\right)\right)\right)\right)\right)$$

A spatial receptive field of 3 x 3 consisting of unit stride (s = 1) and symmetric padding (p = 1) are maintained within every convolutional kernel. The unit padding is crucial for preserving spatial dimensions (H x W) throughout operations by preventing boundary pixel erosion. In order to accelerate gradient descent convergence and regularize internal covariate shift, batch normalization layers are integrated after each convolution. The contracting path (encoder) is primed for extracting hierarchical visual features while widening the network's receptive field. Starting with a single-channel grayscale tensor $X \in R^{(1)(256)(256)}$. The encoder transitions through 4 spatial resolutions.

Step 1: X is projected to f = 16 feature maps at the initial resolution (256 x 256 x 16)
Step 2: A 2 x 2 max-pooling operation with stride 2 halves the spatial resolution, followed by feature expansion to 32 channels (128 x 128 x 32).
Step 3: A second max-pooling layer reduces the map to 64 x 64, expanding channel capacity to 64 channels (64 x 64 x 64).
Step 4 (Bottleneck stage): A final downsampling yields the bottleneck representation at a reduced spatial scale of 32 x 32 with 128 channels (32 x 32 x 128)

The shallow stages complement the deepest bottleneck stage and vice versa by preserving high-frequency gradient transitions and allowing the bottleneck to compress global contextual semantics, allowing the network to distinguish object contours from background textures. The combined abilities both contribute to increased edge detections. The decoder restores low-resolution bottleneck abstractions back to original input resolution. Spatial upsampling is performed using parameter-free bilinear interpolation with a scaling factor of 2, which eliminates checkerboard deconvolution artifacts commonly introduced by transposed convolutions. The terminal layer of the network consists of a 1 x 1 pointwise convolution:

$$\hat{\mathbf{Z}} = \mathrm{Conv}_{1\times1}(\mathbf{F}_{\mathrm{dec}})$$

The operation projects the 16 decoded feature maps down to a single-channel spatial tensor:

$$\hat{\mathbf{Z}} \in \mathbb{R}^{1\times256\times256}$$

Crucially, no non-linear activation is applied at the output head. The model emits unconstrained real-valued logits which are supplied directly to the loss function to maintain numerical stability during training.

IV: Class-Imbalance Objective Function

For our loss function, we contend with an issue that is prominent in edge detection. The issue is that in a standard 256 x 256 image, typically between 5% to 10% of the image would be true edges. With a regular BCE objective function that treats false positives and false negatives as equally bad, the model is incentivized to predict most if not all the pixels as a non-edge as this will still guarantee them a high accuracy in a standard image where most pixels are not an edge. For example, if 90% of an image is a non-edge, the model would get 90% accuracy for predicting every pixel a non-edge. To help address this, in our loss function, we want to punish false negatives more severely as to teach the model to put emphasis on preventing false negatives. Thus, we use a weighted BCE loss function where the contribution to the loss is higher from a false negative.

$$\mathcal{L}_{\text{WBCE}}(\hat{\mathbf{z}}, \mathbf{Y}) = -\frac{1}{N} \sum_{i=1}^{N} [\beta \cdot y_i \log(\sigma(\hat{z}_i)) + (1 - y_i) \log(1 - \sigma(\hat{z}_i))]$$

The original term from regular BCE, $y_i \log(\sigma(\hat{z}_i))$ essentially represents the contribution to the loss when the current target is 1 or an edge in this context. Multiplying this by the weight of β for a positive value, will increase the loss resulting from a false negative, as for a false negative, the target label is 1. β = 5 represents an ideal value in this case for the following reason. Assuming that the ratio of non-edges to edges is approximately 10:1 in a standard image, just choosing to punish false negatives with a value of β = 10, would make the model excessively paranoid leading to more probabilities that result in edge declarations but decreases the precision of the declarations. β = 5 presents a deliberate damped balancing factor that allows for balance between model avoidance of false negatives and maintaining precision for predicted edges.

IV: Decision Threshold Calibration

The decision threshold is a hyperparameter that determines at what confidence level a model has of a pixel being an edge should we start classifying these pixels as edges for confidence levels at the threshold or higher. To determine the optimal threshold for models of both experiments, we test threshold values starting from 0.1 and incrementing by 0.1 until we reach 0.9 on the validation suite. This allows a contamination-free test suite that ultimately can give us an accurate performance estimate on the entire fixed models in the end. The best threshold values determined were τ = 0.7 for the model trained via raw input and τ = 0.6 for the model trained via Laplacian input.

V: Experiments and Analysis

V: Experimental Setup and Implementation Details

To ensure consistency, all experiments were conducted on the benchmark Berkley Segmentation Dataset (BSDS500), where the official dataset partitioning was strictly maintained. This also facilitates comparison and reinforces academic reporting standards. The partitioning is with respect to standard machine learning practice with a training set that consists of 100 images, a validation set that consists of 100 images, and a test set consisting of 200 images. In the case of the training set, any experimentation and model training used a pseudo-label generated by a fixed Canny algorithm for supervised learning as opposed to the ground-truth human-annotated label. The validation set was used exclusively for epoch loss monitoring and decision threshold calibration. The test set was held out for blind evaluation against human consensus ground-truth.

All images were standardized to a spatial resolution of 256 x 256 pixels. Models were implemented in PyTorch, and trained on CPU using Adam optimizer with an initial learning rate of $\eta = 10^{-3}$, weight decay of 10^{-5}, and mini-batch sizes of 8 over 20 epochs. The objective function was configured with a positive class weight of $\beta = 5$ to mitigate the $\approx 10:1$ background-to-edge pixel imbalance. In the case of Laplacian enhancement, the high-boost coefficient was fixed at $\alpha = 0.5$. Explicit Gaussian pre-smoothing was employed for Canny generation of pseudo-labels and was parameterized with $\kappa = 7$ and $\sigma = 1.5$. Hysteresis thresholds of $T_{low} = 100$, and $T_{high} = 200$ were used for ultimate edge identification.

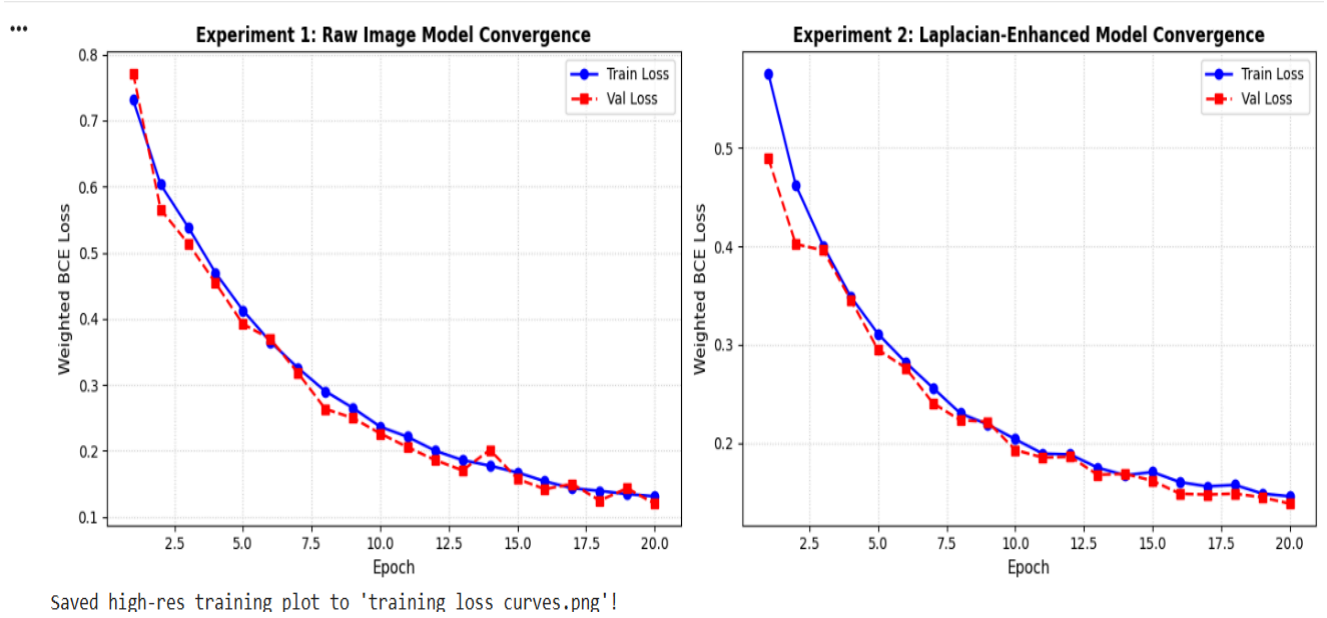


Fig. 4. Training curves of the two models corresponding to the two experiments showcasing low overfitting

V: Evaluation Protocols and Metrics

We evaluate edge detection accuracy by first binarizing our predicted edge maps and comparing against human consensus. However, in the test set of BSDS500, each image corresponds to multiple human-annotated results. Thus, we combine each annotated label into a singular benchmark per image to use for comparison by thresholding the annotator average at > 0.2. Since human hand tracings naturally exhibit slight spatial variation across independent labellers, evaluations were conducted using a standard 1-pixel morphological tolerance window. Under this protocol, edge matches are categorized as follows:

- True Positives (TP): Edge pixels predicted by the model that falls within a 1-pixel radius of an authentic human boundary.
- False Positives (FP): Edge pixels predicted by the model that have no corresponding human boundary within 1-pixel radius
- False Negatives (FP): True human boundary pixels that the model failed to detect within a 1-pixel radius

The F1-score is used as the ultimate quantification for determining performance and is computed as follows.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

V: Quantitative Results and Benchmark Comparison

```
================================================================================
   BSDS500 FINAL BENCHMARK (TUNED ON 100 VAL, EVALUATED ON 200 UNSEEN TEST)
================================================================================
Method / Contender          | Frozen tau*  | Precision  | Recall     | Test F1-Score
--------------------------------------------------------------------------------
1. Classical Canny          | N/A (Fixed)  | 0.3928     | 0.4611     | 0.4242
2. U-Net (Trained on Raw)   | 0.70         | 0.3819     | 0.5176     | 0.4395
3. U-Net (Trained on Lap)   | 0.60         | 0.3982     | 0.5275     | 0.4538
================================================================================
```

Fig. 5. Benchmark for contenders

Both weakly supervised U-Nets were able to outperform their algorithmic teacher. Raw U-Net achieved a +3.6% gain relative to classical Canny edge detection, while U-Net with Laplacian-enhanced input achieved an approximately 7% relative gain. The performance advantage of the neural architectures stems primarily from an improvement in recall, where the models can identify more true edges. In particular with the model trained on Laplacian-enhanced input, there was a 14.4% relative gain with respect to raw Canny's recall score. The implication is that the convolutional network has generalized to an extent such that it learnt to connect broken boundary segments and preserve continuous object contours that Canny's local gradient thresholding terminates prematurely. Pre-sharpening the input via Laplacian produced the best overall performance, having achieved both the highest recall and precision. The amplification of gradient discontinuities streamlined learning for the model and carried an inherent inductive bias which the model can exploit during training.

V: Qualitative Visual Analysis

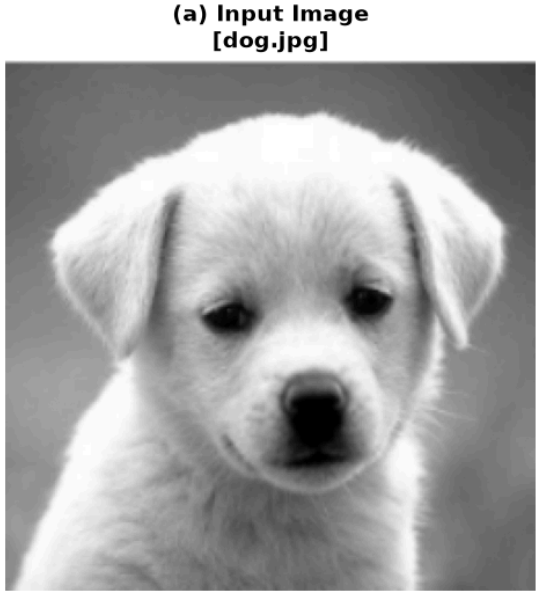


Fig. 6. Grayscale dog image

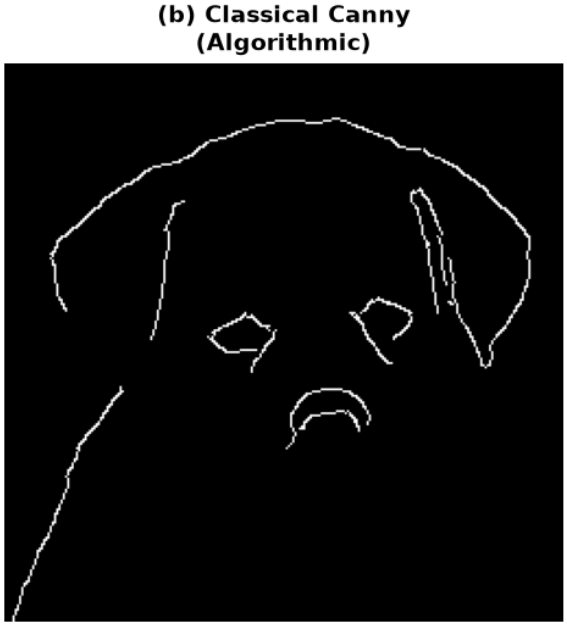


Fig. 7. Classical edge detection

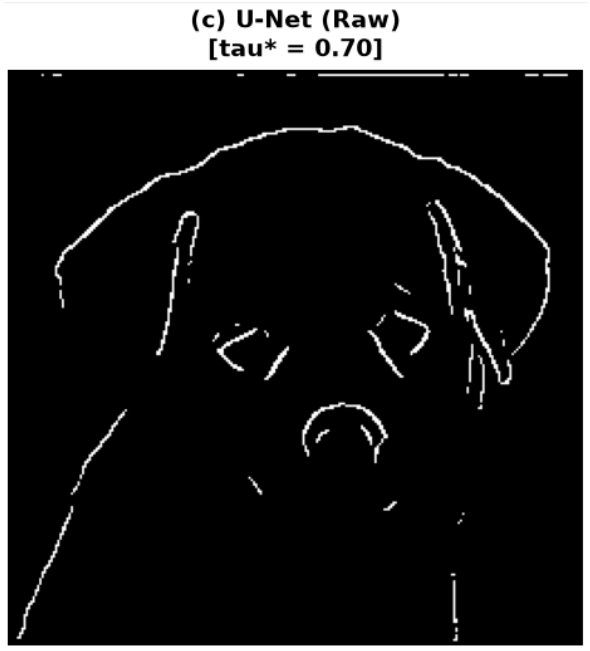


Fig. 8. U-Net trained via raw input edge detection

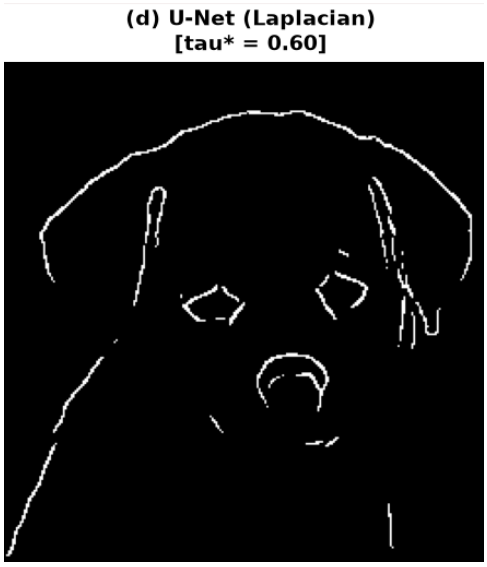


Fig. 9. U-Net trained via Laplacian-enhanced input edge detection

Firstly, we notice that both U-Nets tend predict edges similarly to Canny. This isn't surprising as Canny was used as the supervised labels. However, if we look closer, especially at the Laplacian U-Net, it has generalized beyond Canny, and an example would be the right eye where raw Canny not only failed to close the eye properly, but extended a line downwards, decreasing its precision. However, the Laplacian U-Net did not make the same mistake.

V: Critical Discussion vs Supervised SOTA

One of our central objectives for this experiment was to evaluate how are weakly supervised paradigm compares against fully supervised state-of-the-art architectures to see if it is possible to do away with expensive labelling and achieve results that are still similar to the state-of-the-art. The overall supervised SOTA is EDTER (Edge Detection Transformer, Pu et al., CVPR 2022) with an F1-score of 0.84. The leading CNN-based SOTA is BDCN (Bi-directional Cascade Network, He et al., CVPR 2019) achieving an F1 score of 0.828. Human consensus agreement itself across BSDS500 stands at F1 = 0.803 further demonstrating the prowess of the SOTA edge prediction models. From these results, we can really see the algorithmic limits of our approach, as the human-annotated data allows much better learning of object semantics. Nevertheless, our central hypothesis is validated, as our models have demonstrated that they don't merely just reproduce the noisy algorithmic pseudo-labels and instead are able to smooth out the noise, systematically filter noisy artifacts and outperform their own supervisor.

VI: Conclusion

In this work, we investigated the hypothesis that edge detection models trained on subpar supervised labels can generalize beyond the subpar labels by systematically filtering out noise and keeping the general structure, allowing for higher-quality results. Furthermore, we try to determine how well models trained via this technique can approach fully supervised SOTA. We found that a very modest level of generalization beyond subpar Canny results is absolutely possible even if raw Canny was employed as the supervised label. However, models trained via this approach are not able to remotely approach the level of capability of fully supervised models, as the F1 score of weakly supervised variants (employing Canny supervision), are only a little over half of the F1 score of the overall edge detection SOTA and the convolutional edge detection SOTA.